Rapport de stage

Procédures et fonctions stockées sous MySQL

Base de données **techlearn_db** : gestion de formateurs, stagiaires et sessions de formation

Nom du stagiaire : Chentouf Iyad

Date : 12 novembre 2026

1. Contexte et objectifs pédagogiques

Ce rapport présente l'exercice pratique réalisé sur MySQL autour d'une plateforme de formation fictive, **TechLearn**. La base de données *techlearn_db* regroupe trois entités : les **formateurs** (nom, spécialité), les **stagiaires** (nom, ville) et les **sessions** de formation (intitulé, formateur responsable, capacité d'accueil et nombre d'inscrits). Chaque session est rattachée à un formateur par une clé étrangère.

Objectifs pédagogiques

- Créer une base de données et des tables reliées par clés primaires et étrangères.
- Insérer des jeux de données de test avec `INSERT INTO`.
- Écrire des **procédures stockées** avec les trois modes de paramètres : `IN`, `OUT` et `INOUT`.
- Écrire des **fonctions stockées** qui renvoient une valeur utilisable directement dans une requête `SELECT`.
- Utiliser les structures de contrôle : variables locales (`DECLARE`), boucle `WHILE`, condition `IF / ELSE`.
- Manipuler les données depuis une procédure (`UPDATE`, `DELETE`, `INSERT`) et vérifier les résultats.

2. Code SQL complet

2.1 Création de la base de données

```
CREATE DATABASE techlearn_db;
USE techlearn_db;
```

2.2 Création des tables

```
CREATE TABLE formateurs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100),
  specialite VARCHAR(100)
);

CREATE TABLE stagiaires (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100),
  ville VARCHAR(100)
);

CREATE TABLE sessions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  intitule VARCHAR(100),
  formateur_id INT,
  capacite INT,
  inscrits INT DEFAULT 0,
  FOREIGN KEY (formateur_id) REFERENCES formateurs(id)
);
```

2.3 Insertion des données

```
INSERT INTO formateurs (nom, specialite) VALUES
('Mme Nadia', 'Bases de données'),
('M. Karim', 'Développement Web'),
('Mme Salma', 'Analyse de données');

INSERT INTO stagiaires (nom, ville) VALUES
('Ali', 'Casablanca'),
('Sara', 'Rabat'),
('Mouna', 'Marrakech');

INSERT INTO sessions (intitule, formateur_id, capacite, inscrits) VALUES
('MySQL avancé', 1, 20, 5),
('HTML/CSS', 2, 25, 10),
('Python Data', 3, 15, 8);
```

2.4 Procédure avec paramètre IN : afficher_stagiaires_par_ville

```
DELIMITER //
CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom VARCHAR(100))
BEGIN
  SELECT * FROM stagiaires WHERE ville = ville_nom;
END //
DELIMITER ;

CALL afficher_stagiaires_par_ville('Casablanca');
```

2.5 Procédure avec paramètre OUT : compter_sessions

```
DELIMITER //
CREATE PROCEDURE compter_sessions(OUT total INT)
BEGIN
  SELECT COUNT(*) INTO total FROM sessions;
END //
DELIMITER ;

CALL compter_sessions(@nb);
SELECT @nb AS nombre_total_sessions;
```

2.6 Procédure avec paramètre INOUT : inscrire_stagiaire

```
DELIMITER //
```

```
CREATE PROCEDURE inscrire_stagiaire(INOUT nb_inscrits INT, IN session_id INT)
BEGIN
  UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
  SELECT inscrits INTO nb_inscrits FROM sessions WHERE id = session_id;
END //
DELIMITER ;

SET @nouveau = 0;
CALL inscrire_stagiaire(@nouveau, 1);
SELECT @nouveau AS nouveau_nombre_inscrits;
```

2.7 Fonction : taux_occupation

```
DELIMITER //
CREATE FUNCTION taux_occupation(id_session INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE taux DECIMAL(5,2);
  SELECT (inscrits / capacite) * 100 INTO taux
  FROM sessions WHERE id = id_session;
  RETURN taux;
END //
DELIMITER ;

SELECT intitule, taux_occupation(id) AS taux FROM sessions;
```

2.8 Procédure avec boucle WHILE : generer_sessions

```
DELIMITER //
CREATE PROCEDURE generer_sessions(IN formateur INT)
BEGIN
  DECLARE i INT DEFAULT 1;
  WHILE i <= 10 DO
    INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
    VALUES (CONCAT('Session ', i, ' - Formateur ', formateur), formateur, 20, 0);
    SET i = i + 1;
  END WHILE;
END //
DELIMITER ;

CALL generer_sessions(2);
SELECT * FROM sessions;
```

2.9 Fonction avec jointure : sessions_formateur

```
DELIMITER //
CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE nb INT;
  SELECT COUNT(*) INTO nb
  FROM sessions s
  JOIN formateurs f ON s.formateur_id = f.id
  WHERE f.nom = nom_formateur;
  RETURN nb;
END //
DELIMITER ;

SELECT sessions_formateur('M. Karim');
```

2.10 Procédure de nettoyage : supprimer_sessions_vides

```
DELIMITER //
CREATE PROCEDURE supprimer_sessions_vides()
BEGIN
  DELETE FROM sessions WHERE inscrits = 0;
END //
DELIMITER ;
```

```
CALL supprimer_sessions_vides();
SELECT * FROM sessions;
```

2.11 Procédure avec condition IF : valider_inscription

```
DELIMITER //
CREATE PROCEDURE valider_inscription(IN session_id INT)
BEGIN
  DECLARE v_inscrits INT;
  DECLARE v_capacite INT;

  SELECT inscrits, capacite INTO v_inscrits, v_capacite
  FROM sessions WHERE id = session_id;

  IF v_inscrits >= v_capacite THEN
    SELECT 'Session complète' AS message;
  ELSE
    UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    SELECT 'Inscription validée' AS message;
  END IF;
END //
DELIMITER ;

CALL valider_inscription(1);

-- Test du cas "session complète"
UPDATE sessions SET inscrits = capacite WHERE id = 3;
CALL valider_inscription(3);
```

3. Captures d'écran et analyse des résultats

Chaque capture montre l'exécution réelle des requêtes dans le client MySQL en ligne de commande. L'analyse qui suit explique ce que fait chaque procédure ou fonction et interprète le résultat obtenu.

3.1 Création de la base de données

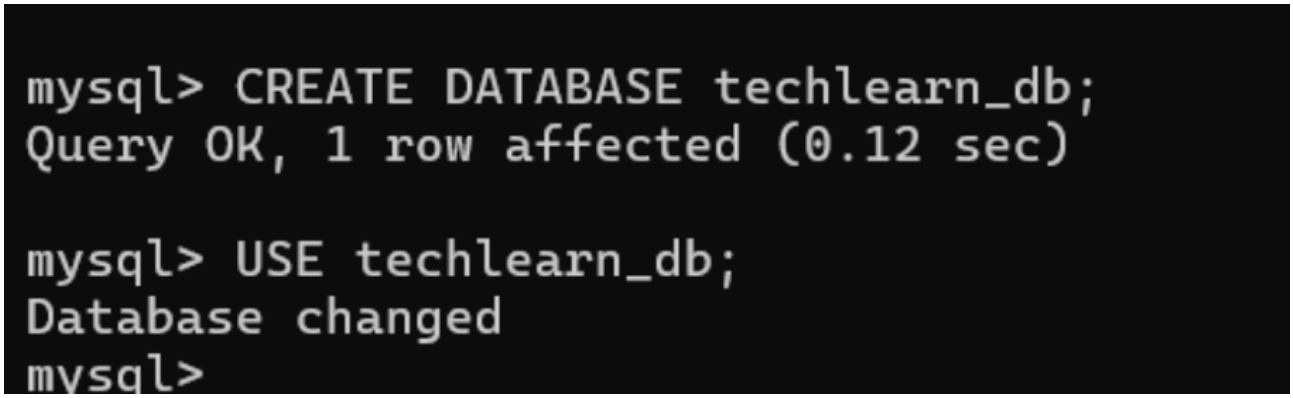


Figure 1 – Création et sélection de la base techlearn_db

La commande `CREATE DATABASE` répond « Query OK, 1 row affected » : la base est créée. `USE` la sélectionne (« Database changed »), ce qui garantit que toutes les tables et routines suivantes lui seront rattachées.

3.2 Création des tables

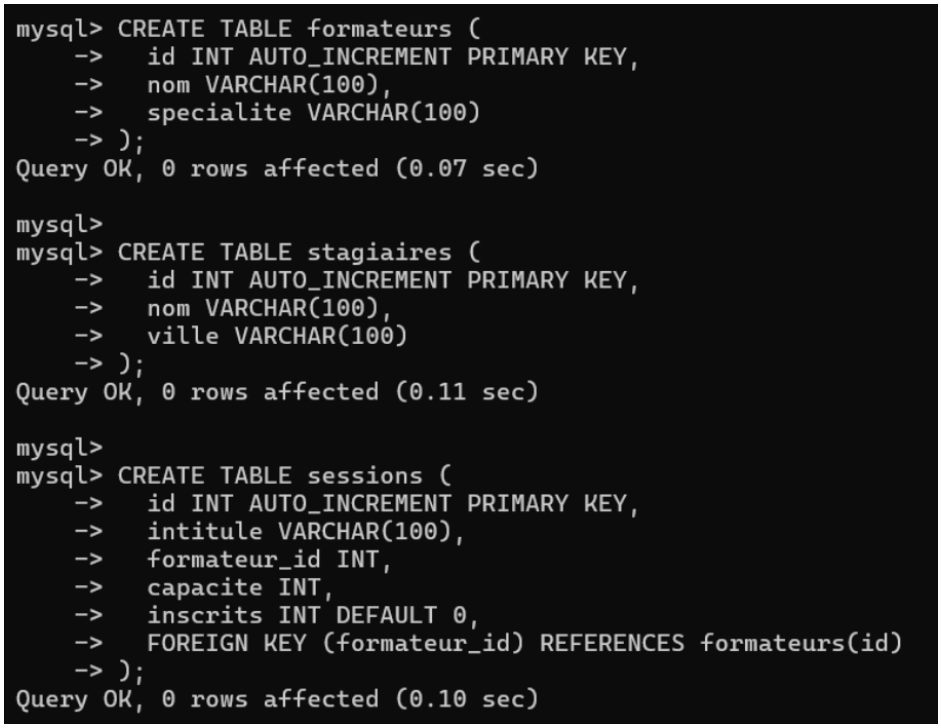


Figure 2 – Création des tables formateurs, stagiaires et sessions

Les trois tables sont créées sans erreur. Les clés primaires `AUTO_INCREMENT` génèrent automatiquement les identifiants. La table *sessions* contient une clé étrangère vers *formateurs(id)* : elle empêche d'affecter une session à un formateur inexistant. La valeur par défaut `inscrits INT DEFAULT 0` initialise le compteur d'inscrits.

3.3 Insertion des données

```
mysql> INSERT INTO formateurs (nom, specialite) VALUES
    -> ('Mme Nadia', 'Bases de données'),
    -> ('M. Karim', 'Développement Web'),
    -> ('Mme Salma', 'Analyse de données');
Query OK, 3 rows affected (0.07 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO stagiaires (nom, ville) VALUES
    -> ('Ali', 'Casablanca'),
    -> ('Sara', 'Rabat'),
    -> ('Mouna', 'Marrakech');
Query OK, 3 rows affected (0.37 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO sessions (intitule, formateur_id, capacite, inscrits) VALUES
    -> ('MySQL avancé', 1, 20, 5),
    -> ('HTML/CSS', 2, 25, 10),
    -> ('Python Data', 3, 15, 8);
Query OK, 3 rows affected (0.07 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Figure 3 – Insertion des données de test

Chaque `INSERT` affiche « Records: 3 Duplicates: 0 Warnings: 0 » : trois formateurs, trois stagiaires et trois sessions ont été insérés sans doublon ni avertissement. Les valeurs initiales (5, 10 et 8 inscrits) serviront de référence pour les tests suivants.

3.4 Paramètre IN

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom VARCHAR(100))
    -> BEGIN
    ->   SELECT * FROM stagiaires WHERE ville = ville_nom;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> CALL afficher_stagiaires_par_ville('Casablanca');
+----+------+------------+
| id | nom  | ville      |
+----+------+------------+
|  1 | Ali  | Casablanca |
+----+------+------------+
1 row in set (0.07 sec)

Query OK, 0 rows affected (0.07 sec)
```

Figure 4 – Procédure afficher_stagiaires_par_ville (paramètre IN)

Le paramètre `IN` transmet une valeur à la procédure sans la modifier. L'appel avec « Casablanca » renvoie un seul stagiaire, Ali. L'instruction `DELIMITER //` est nécessaire pour que le `;` interne au corps de la procédure ne termine pas prématurément la commande de création.

3.5 Paramètre OUT

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE compter_sessions(OUT total INT)
    -> BEGIN
    ->   SELECT COUNT(*) INTO total FROM sessions;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> CALL compter_sessions(@nb);
Query OK, 1 row affected (0.06 sec)

mysql> SELECT @nb AS nombre_total_sessions;
+-----------------------+
| nombre_total_sessions |
+-----------------------+
|                     3 |
+-----------------------+
1 row in set (0.07 sec)
```

Figure 5 – Procédure compter_sessions (paramètre OUT)

Avec `OUT`, la procédure renvoie une valeur à l'appelant via la variable de session `@nb`. Le résultat affiché, **3**, correspond bien aux trois sessions insérées à l'étape précédente.

3.6 Paramètre INOUT

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE inscrire_stagiaire(INOUT nb_inscrits INT, IN session_id INT)
    -> BEGIN
    ->   UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    ->   SELECT inscrits INTO nb_inscrits FROM sessions WHERE id = session_id;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> SET @nouveau = 0;
Query OK, 0 rows affected (0.06 sec)

mysql> CALL inscrire_stagiaire(@nouveau, 1);
Query OK, 1 row affected (0.07 sec)

mysql> SELECT @nouveau AS nouveau_nombre_inscrits;
+-------------------------+
| nouveau_nombre_inscrits |
+-------------------------+
|                       6 |
+-------------------------+
1 row in set (0.06 sec)
```

Figure 6 – Procédure inscrire_stagiaire (paramètre INOUT)

Un paramètre `INOUT` sert à la fois d'entrée et de sortie. La session 1 passe de 5 à **6** inscrits : la procédure met à jour la table puis relit la nouvelle valeur dans `@nouveau`. Cette valeur de 6 inscrits est celle que l'on retrouve dans les captures suivantes.

3.7 Fonction de calcul

```
mysql> DELIMITER //
mysql> CREATE FUNCTION taux_occupation(id_session INT)
    -> RETURNS DECIMAL(5,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE taux DECIMAL(5,2);
    ->   SELECT (inscrits / capacite) * 100 INTO taux
    ->   FROM sessions WHERE id = id_session;
    ->   RETURN taux;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> SELECT intitule, taux_occupation(id) AS taux FROM sessions;
+--------------+-------+
| intitule     | taux  |
+--------------+-------+
| MySQL avancé | 30.00 |
| HTML/CSS     | 40.00 |
| Python Data  | 53.33 |
+--------------+-------+
3 rows in set (0.10 sec)
```

Figure 7 – Fonction taux_occupation

Contrairement à une procédure, une fonction retourne une valeur et s'emploie directement dans un `SELECT`. Les taux obtenus sont cohérents : MySQL avancé 6/20 = **30,00 %**, HTML/CSS 10/25 = **40,00 %**, Python Data 8/15 = **53,33 %**. Le type `DECIMAL(5,2)` limite l'affichage à deux décimales.

3.8 Boucle WHILE

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE generer_sessions(IN formateur INT)
    -> BEGIN
    ->   DECLARE i INT DEFAULT 1;
    ->   WHILE i <= 10 DO
    ->     INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
    ->     VALUES (CONCAT('Session ', i, ' - Formateur ', formateur), formateur, 20, 0);
    ->     SET i = i + 1;
    ->   END WHILE;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> CALL generer_sessions(2);
Query OK, 1 row affected (0.10 sec)

mysql> SELECT * FROM sessions;
+----+--------------------------+--------------+----------+----------+
| id | intitule                 | formateur_id | capacite | inscrits |
+----+--------------------------+--------------+----------+----------+
|  1 | MySQL avancé             |            1 |       20 |        6 |
|  2 | HTML/CSS                 |            2 |       25 |       10 |
|  3 | Python Data              |            3 |       15 |        8 |
|  4 | Session 1 - Formateur 2  |            2 |       20 |        0 |
|  5 | Session 2 - Formateur 2  |            2 |       20 |        0 |
|  6 | Session 3 - Formateur 2  |            2 |       20 |        0 |
|  7 | Session 4 - Formateur 2  |            2 |       20 |        0 |
|  8 | Session 5 - Formateur 2  |            2 |       20 |        0 |
|  9 | Session 6 - Formateur 2  |            2 |       20 |        0 |
| 10 | Session 7 - Formateur 2  |            2 |       20 |        0 |
| 11 | Session 8 - Formateur 2  |            2 |       20 |        0 |
| 12 | Session 9 - Formateur 2  |            2 |       20 |        0 |
| 13 | Session 10 - Formateur 2 |            2 |       20 |        0 |
+----+--------------------------+--------------+----------+----------+
13 rows in set (0.07 sec)
```

Figure 8 – Procédure generer_sessions (boucle WHILE)

La boucle insère dix sessions pour le formateur 2, numérotées de 1 à 10, chacune avec une capacité de 20 et zéro inscrit. Le `SELECT * ` affiche désormais **13 lignes** (3 initiales + 10 générées), ce qui confirme que la boucle s'est exécutée le nombre de fois prévu.

3.9 Fonction avec jointure

```
mysql> DELIMITER //
mysql> CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
    -> RETURNS INT
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE nb INT;
    ->   SELECT COUNT(*) INTO nb
    ->   FROM sessions s
    ->   JOIN formateurs f ON s.formateur_id = f.id
    ->   WHERE f.nom = nom_formateur;
    ->   RETURN nb;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> SELECT sessions_formateur('M. Karim');
+-------------------------------+
| sessions_formateur('M. Karim') |
+-------------------------------+
|                            11 |
+-------------------------------+
1 row in set (0.07 sec)
```

Figure 9 – Fonction sessions_formateur

La fonction joint *sessions* et *formateurs* pour compter les sessions d'un formateur donné. Pour M. Karim, le résultat est **11** : sa session « HTML/CSS » plus les 10 sessions générées à l'étape précédente. Ce chiffre valide à la fois la fonction et la procédure précédente.

3.10 Suppression conditionnelle

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE supprimer_sessions_vides()
    -> BEGIN
    ->   DELETE FROM sessions WHERE inscrits = 0;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> CALL supprimer_sessions_vides();
Query OK, 10 rows affected (0.07 sec)

mysql> SELECT * FROM sessions;
+----+--------------+--------------+----------+----------+
| id | intitule     | formateur_id | capacite | inscrits |
+----+--------------+--------------+----------+----------+
|  1 | MySQL avancé |            1 |       20 |        6 |
|  2 | HTML/CSS     |            2 |       25 |       10 |
|  3 | Python Data  |            3 |       15 |        8 |
+----+--------------+--------------+----------+----------+
3 rows in set (0.07 sec)
```

Figure 10 – Procédure supprimer_sessions_vides

L'appel affiche « 10 rows affected » : les 10 sessions générées, qui n'avaient aucun inscrit, ont été supprimées. Il ne reste que les 3 sessions d'origine, toutes avec des inscrits. La procédure est utile pour nettoyer la table, mais elle est irréversible : en production, il faudrait l'utiliser avec précaution, par exemple dans une transaction.

3.11 Condition IF / ELSE

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE valider_inscription(IN session_id INT)
    -> BEGIN
    ->   DECLARE v_inscrits INT;
    ->   DECLARE v_capacite INT;
    ->
    ->   SELECT inscrits, capacite INTO v_inscrits, v_capacite
    ->   FROM sessions WHERE id = session_id;
    ->
    ->   IF v_inscrits >= v_capacite THEN
    ->     SELECT 'Session complète' AS message;
    ->   ELSE
    ->     UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    ->     SELECT 'Inscription validée' AS message;
    ->   END IF;
    -> END //
Query OK, 0 rows affected (0.07 sec)

mysql> DELIMITER ;
mysql>
mysql> CALL valider_inscription(1);
+--------------------+
| message            |
+--------------------+
| Inscription validée |
+--------------------+
1 row in set (0.07 sec)
```

Figure 11 – Procédure valider_inscription : cas d'une inscription acceptée

La session 1 compte 6 inscrits pour une capacité de 20 : la condition `inscrits >= capacite` est fausse, la branche `ELSE` s'exécute, le compteur passe à 7 et le message « Inscription validée » est renvoyé.

3.12 Test de la limite

```
mysql> UPDATE sessions SET inscrits = capacite WHERE id = 3;
Query OK, 1 row affected (0.07 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> CALL valider_inscription(3);   -- Session complète
+------------------+
| message          |
+------------------+
| Session complète |
+------------------+
1 row in set (0.07 sec)
```

Figure 12 – Procédure valider_inscription : cas d'une session complète

Pour tester le cas limite, la session 3 est d'abord remplie (`inscrits = capacite`). L'appel de la procédure déclenche cette fois la branche `THEN` et affiche « Session complète » sans modifier la table. Les deux cas du `IF` ont donc été testés.

4. Conclusion personnelle

Cet exercice m'a permis de passer de simples requêtes SQL à une logique métier écrite directement dans la base de données. J'ai compris la différence entre une **procédure**, que l'on appelle avec `CALL` et qui peut modifier des données ou renvoyer plusieurs résultats, et une **fonction**, qui retourne une

seule valeur et s'utilise dans une requête comme n'importe quelle expression. Les trois modes de paramètres (`IN`, `OUT`, `INOUT`) m'ont aussi permis de comprendre comment une routine échange des informations avec l'appelant.

J'ai également appris à utiliser les variables locales, la boucle `WHILE` et la condition `IF / ELSE`, ainsi que l'importance de `DELIMITER` pour définir des routines. Le fait d'enchaîner les tests (par exemple le total de 11 sessions pour M. Karim, puis la suppression des 10 sessions vides) m'a montré qu'il faut toujours vérifier l'état des données après chaque appel.

Enfin, j'ai identifié des pistes d'amélioration : la procédure *inscrire_stagiaire* ne vérifie pas la capacité maximale (contrairement à *valider_inscription*), la fonction *taux_occupation* ne gère pas une capacité égale à zéro (division par zéro), et les suppressions devraient être protégées par une transaction. Centraliser ces règles dans la base garantit la cohérence des données et évite de dupliquer le code dans chaque application.